ENGR 4399 · CYBER-PHYSICAL & IOT SYSTEMS · PROJECT 2

Mesh Barista

A distributed, resilient pump control system that pours a measured cup of coffee, milk and cream.

TEAM

Sebastian Cadena · Daniel Critchlow Jr

November 12, 2026

Problem and motivation

A pump that keeps running after it loses contact with its tank wastes water and destroys itself.

Project 1 put every decision in one ESP32 and a cloud API. Real water systems are spread out, and nodes fail without warning.

Dry-running pumps

An empty source tank burns out the pump.

Overflow

A full destination tank means water on the floor.

Silent failures

Operators cannot tell which zone went down.

Single point of failure

One controller or router takes the whole system with it.

Our model of it: three reservoirs are the source tanks, the cup is the destination tank, and dyed water stands in for the drinks.

System architecture

NODE 1 · SOURCE / PUMP

Pump station

3 reservoir level sensors

3 relays driving 3 pumps

Buzzer

Own fail-safe: pumps off



NODE 2 · GATEWAY

Control center

Joystick menu and OLED

Buzzer alarm

USB serial dashboard

Relays Node 1 and Node 3



NODE 3 · DISTRIBUTION

Output

Cup level sensor

Status OLED

RGB LED: pump, ready, error

Its reading stops the pump

OPERATOR VIEW

Every node is a Wi-Fi access point that hosts the same web dashboard, so it survives the loss of any one node.

No router, cloud service or API is needed: each node that owns an actuator decides locally.

Technical implementation

WIRELESS

ESP-NOW with our own relay

Broadcast frames: no router, no pairing.

Sequence numbers drop repeats.

A node rebroadcasts what it hears once, so a node out of range is still reached.

Heartbeat every second; lost after 3 s.

DATA

JSON messages

```
{"node":1,"seq":145,
 "zone":"pump station",
 "up":80925,"pump":0,
 "lvl":[77,50,77],
 "state":"SAFE",
 "why":"lost Node 2"}
```

Largest message is about 175 bytes; one frame holds 250.

INTERFACE

Dashboard on every node

Each ESP32 serves the page and a JSON status feed over HTTP.

Shows nodes alive or lost, levels, pump state, event log and mesh paths.

Test buttons run pumps, buzzers and the LED.

Sustainability logic

RULE	THRESHOLD	WHAT IT PROTECTS
Dry-run lock	below 10 %, unlock above 20 %	The pump; the gap stops relay chatter
Overflow stop	cup at 90 %, read by Node 3	No spilled water
Stale data stop	no cup reading for 1.5 s	Never pour blind
Lost node	no heartbeat for 3 s	Pumps off, buzzer, zone flagged
Leave SAFE	operator acknowledge	A half-poured cup is an unknown state

NORMAL

All nodes alive. Pours allowed.

DEGRADED

A reservoir is low, or running without Node 2.

SAFE

Pumps off, buzzers on, LED flashing red.

Results and demonstration



Pick a drink on the joystick or dashboard, pour by time, stop on a full cup, then pull a node and watch SAFE.

18 / 18

control and fault scenarios passed in simulation, including each node lost in turn

3 s

to flag a lost node and name its zone on the dashboard

[_ _] mL

measured error on a 60 mL pour, hardware

[_ _] mL/s

measured pump flow rate used for timed pours

Lessons learned and improvements

WHAT WAS HARD

Engineering challenges

A full mesh library was more than three nodes needed; ESP-NOW with a small relay was simpler to reason about.

A clicking relay does not prove the pump has power: the pump circuit needs its own test.

A joystick does not rest at mid-scale, so it is calibrated at boot.

Water sensors are not linear, so they act as limits, not measurements.

WHAT COMES NEXT

Future enhancements

Load cell under the cup to pour by weight.

Servo pinch valve so SAFE also closes the outlet.

Flow check that catches a blocked tube or empty line.

Encrypted ESP-NOW and a login on the dashboard.